In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df = pd.read_csv("/content/retail_orders_2026.csv")

print(df)
```

```
       ORDER_ID  ORDER_DATE CUSTOMER_ID  CUSTOMER_NAME PRODUCT_ID  \
0    ORD2600019  2026-01-04       C0007    Diya Kapoor      P3001
1    ORD2600141  2026-01-04       C0018    Charan Jain      P3002
2    ORD2600404  2026-01-04       C0060  Naveen Sharma      P1003
3    ORD2600225  2026-01-05       C0011   Vidhya Menon      P1006
4    ORD2600236  2026-01-05       C0047    Diya Kapoor      P3006
..          ...         ...         ...            ...        ...
535  ORD2600045  2026-10-25       C0118     Rahul Jain      P4005
536  ORD2600362  2026-10-25       C0073   Sanjay Mehta      P4004
537  ORD2600484  2026-10-25       C0086     Akash Iyer      P4004
538  ORD2600369  2026-10-26       C0085     Tara Singh      P2005
539  ORD2600198  2026-10-28       C0112  Pranav Sundar      P4003

                     PRODUCT          CATEGORY  QUANTITY  UNIT_PRICE  \
0      Stainless Steel Bottle    Home & Kitchen         1       899.0
1                  Air Fryer    Home & Kitchen         1      6999.0
2          Bluetooth Speaker       Electronics         1      2499.0
3    Noise Cancelling Earbuds       Electronics         2      5499.0
4                Coffee Maker    Home & Kitchen         2      3899.0
..                        ...               ...       ...         ...
535          Resistance Bands  Sports & Fitness         1       749.0
536         Tennis Balls Pack  Sports & Fitness         1       649.0
537         Tennis Balls Pack  Sports & Fitness         1       649.0
538               Linen Shirt           Fashion         1      1499.0
539          Badminton Racket  Sports & Fitness         2      2299.0

     DISCOUNT_PCT        CITY        STATE PAYMENT_METHOD SALES_CHANNEL  \
0             0.0  Coimbatore   Tamil Nadu           UPI    MOBILE_APP
1            10.0       Delhi        Delhi    DEBIT_CARD    MOBILE_APP
2             0.0   Hyderabad    Telangana   CREDIT_CARD    MOBILE_APP
3             0.0   Hyderabad    Telangana           UPI         STORE
4             0.0  Coimbatore   Tamil Nadu   NET_BANKING    MOBILE_APP
..            ...         ...          ...           ...           ...
535           5.0   Bengaluru    Karnataka    DEBIT_CARD    MOBILE_APP
536           5.0   Bengaluru    Karnataka           UPI    MOBILE_APP
537          25.0      Mumbai  Maharashtra   CREDIT_CARD    MOBILE_APP
538           0.0        Pune  Maharashtra           COD         STORE
539          15.0   Bengaluru    Karnataka           UPI       WEBSITE

    ORDER_STATUS  SHIPPING_FEE CUSTOMER_SEGMENT
0      DELIVERED          79.0           SILVER
1        SHIPPED           0.0          REGULAR
2     PROCESSING           0.0          REGULAR
3        SHIPPED           0.0             GOLD
4      DELIVERED           0.0             GOLD
..           ...           ...              ...
535   PROCESSING          39.0           SILVER
536      SHIPPED          79.0          REGULAR
537    DELIVERED          39.0          REGULAR
538    DELIVERED          79.0          STUDENT
539    DELIVERED           0.0           SILVER

[540 rows x 17 columns]
```

In [3]:
```
print("First 5 rows:")
print(df.head())
```

```
First 5 rows:
     ORDER_ID  ORDER_DATE CUSTOMER_ID  CUSTOMER_NAME PRODUCT_ID  \
0  ORD2600019  2026-01-04       C0007    Diya Kapoor      P3001
1  ORD2600141  2026-01-04       C0018    Charan Jain      P3002
2  ORD2600404  2026-01-04       C0060  Naveen Sharma      P1003
3  ORD2600225  2026-01-05       C0011   Vidhya Menon      P1006
4  ORD2600236  2026-01-05       C0047    Diya Kapoor      P3006

                   PRODUCT        CATEGORY  QUANTITY  UNIT_PRICE  \
0    Stainless Steel Bottle  Home & Kitchen         1       899.0
1                Air Fryer  Home & Kitchen         1      6999.0
2         Bluetooth Speaker     Electronics         1      2499.0
3  Noise Cancelling Earbuds     Electronics         2      5499.0
4              Coffee Maker  Home & Kitchen         2      3899.0

   DISCOUNT_PCT        CITY       STATE PAYMENT_METHOD SALES_CHANNEL  \
0           0.0  Coimbatore  Tamil Nadu            UPI    MOBILE_APP
1          10.0       Delhi       Delhi     DEBIT_CARD    MOBILE_APP
2           0.0   Hyderabad   Telangana    CREDIT_CARD    MOBILE_APP
3           0.0   Hyderabad   Telangana            UPI         STORE
4           0.0  Coimbatore  Tamil Nadu    NET_BANKING    MOBILE_APP

  ORDER_STATUS  SHIPPING_FEE CUSTOMER_SEGMENT
0    DELIVERED          79.0           SILVER
1      SHIPPED           0.0          REGULAR
2   PROCESSING           0.0          REGULAR
3      SHIPPED           0.0             GOLD
4    DELIVERED           0.0             GOLD
```

In [4]:
```
print("Last 5 rows:")
print(df.tail())
```

```
Last 5 rows:
       ORDER_ID  ORDER_DATE CUSTOMER_ID  CUSTOMER_NAME PRODUCT_ID  \
535  ORD2600045  2026-10-25       C0118     Rahul Jain      P4005
536  ORD2600362  2026-10-25       C0073   Sanjay Mehta      P4004
537  ORD2600484  2026-10-25       C0086     Akash Iyer      P4004
538  ORD2600369  2026-10-26       C0085     Tara Singh      P2005
539  ORD2600198  2026-10-28       C0112  Pranav Sundar      P4003

                PRODUCT          CATEGORY  QUANTITY  UNIT_PRICE  DISCOUNT_PCT  \
535   Resistance Bands  Sports & Fitness         1       749.0           5.0
536  Tennis Balls Pack  Sports & Fitness         1       649.0           5.0
537  Tennis Balls Pack  Sports & Fitness         1       649.0          25.0
538        Linen Shirt           Fashion         1      1499.0           0.0
539   Badminton Racket  Sports & Fitness         2      2299.0          15.0

          CITY        STATE PAYMENT_METHOD SALES_CHANNEL ORDER_STATUS  \
535  Bengaluru    Karnataka     DEBIT_CARD    MOBILE_APP   PROCESSING
536  Bengaluru    Karnataka            UPI    MOBILE_APP      SHIPPED
537     Mumbai  Maharashtra    CREDIT_CARD    MOBILE_APP    DELIVERED
538       Pune  Maharashtra            COD         STORE    DELIVERED
539  Bengaluru    Karnataka            UPI       WEBSITE    DELIVERED

     SHIPPING_FEE CUSTOMER_SEGMENT
535          39.0           SILVER
536          79.0          REGULAR
537          39.0          REGULAR
538          79.0          STUDENT
539           0.0           SILVER
```

In [5]:
```
print("Dataset information:")
df.info()
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 540 entries, 0 to 539
Data columns (total 17 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   ORDER_ID          540 non-null    object 
 1   ORDER_DATE        540 non-null    object 
 2   CUSTOMER_ID       540 non-null    object 
 3   CUSTOMER_NAME     540 non-null    object 
 4   PRODUCT_ID        540 non-null    object 
 5   PRODUCT           540 non-null    object 
 6   CATEGORY          540 non-null    object 
 7   QUANTITY          540 non-null    int64  
 8   UNIT_PRICE        540 non-null    float64
 9   DISCOUNT_PCT      534 non-null    float64
 10  CITY              533 non-null    object 
 11  STATE             540 non-null    object 
 12  PAYMENT_METHOD    540 non-null    object 
 13  SALES_CHANNEL     540 non-null    object 
 14  ORDER_STATUS      540 non-null    object 
 15  SHIPPING_FEE      540 non-null    float64
 16  CUSTOMER_SEGMENT  540 non-null    object 
dtypes: float64(3), int64(1), object(13)
memory usage: 71.8+ KB
```

In [6]:
```
print("Dataset shape:")
print(df.shape)
```

```
Dataset shape:
(540, 17)
```

In [7]:
```
print("Dataset shape:")
print(df.shape)
```

```
Dataset shape:
(540, 17)
```

In [8]:
```
print("Column names:")
print(df.columns)
```

```
Column names:
Index(['ORDER_ID', 'ORDER_DATE', 'CUSTOMER_ID', 'CUSTOMER_NAME', 'PRODUCT_ID',
       'PRODUCT', 'CATEGORY', 'QUANTITY', 'UNIT_PRICE', 'DISCOUNT_PCT', 'CITY',
       'STATE', 'PAYMENT_METHOD', 'SALES_CHANNEL', 'ORDER_STATUS',
       'SHIPPING_FEE', 'CUSTOMER_SEGMENT'],
      dtype='object')
```

In [9]:
```
print(df.describe())
```

```
         QUANTITY   UNIT_PRICE  DISCOUNT_PCT  SHIPPING_FEE
count  540.000000   540.000000    534.000000    540.000000
mean     1.529630  2084.277778      6.235955     21.501852
std      0.842434  1777.271187      7.042390     28.020699
min      1.000000   299.000000      0.000000      0.000000
25%      1.000000   749.000000      0.000000      0.000000
50%      1.000000  1499.000000      5.000000      0.000000
75%      2.000000  2849.000000     10.000000     39.000000
max      5.000000  6999.000000     25.000000     79.000000
```

In [10]:
```
print("Missing values in each column:")
print(df.isnull().sum())
```

```
Missing values in each column:
ORDER_ID            0
ORDER_DATE          0
CUSTOMER_ID         0
CUSTOMER_NAME       0
PRODUCT_ID          0
PRODUCT             0
CATEGORY            0
QUANTITY            0
UNIT_PRICE          0
DISCOUNT_PCT        6
CITY                7
STATE               0
PAYMENT_METHOD      0
SALES_CHANNEL       0
ORDER_STATUS        0
SHIPPING_FEE        0
CUSTOMER_SEGMENT    0
dtype: int64
```

In [11]:
```
print("Total missing values:")
print(df.isnull().sum().sum())
```

```
Total missing values:
13
```

In [12]:
```
print("Number of duplicate rows:")
print(df.duplicated().sum())
```

```
Number of duplicate rows:
0
```

In [13]:
```
print("Create a NumPy array from a column")
unit_price = df["UNIT_PRICE"].to_numpy()

print("NumPy array:")
print(unit_price)
```

```
Create a NumPy array from a column
NumPy array:
[ 899. 6999. 2499. 5499. 3899.  399. 1599.  799. 4999.  649. 1899. 3899.
 1899. 2299. 2199.  749. 6999.  799. 2299.  299. 3299. 3299.  649. 2699.
 3299.  749. 6999. 5499. 1499.  799. 6999. 3899.  299. 3299.  499.  299.
  899. 5499.  399. 2499. 6999. 1599. 1999.  899. 4999. 4999.  749. 1999.
 2199. 6999. 1499.  299.  999. 4999.  999.  299. 2699.  599. 3499.  599.
  299. 1499.  499.  899. 3299. 2499.  799.  399. 4999. 1599.  299. 4999.
  799. 2299.  799. 1499.  299. 1499. 3499.  649.  349.  649. 2199.  899.
  299. 2699.  499. 2299. 3899. 1899. 6999. 1999. 6999.  899. 3899. 1499.
 1499. 1999. 2199.  499. 1499.  299. 2699. 5499. 1299. 1599.  499.  649.
 2199. 2699. 1499.  349. 5499. 3299. 2499. 2699. 3299. 2299.  899.  349.
 1599. 3899. 5499.  899.  749. 3299. 3499. 5499. 2499. 1999.  349.  499.
  499.  749. 2699. 5499.  499.  899. 2699.  649. 1899. 1999.  999.  499.
 2499. 3299.  599.  349.  799.  799. 3499.  299.  349. 4999. 3899.  899.
  499. 1899. 1499.  999. 6999. 1499.  499. 1499. 4999.  999.  499. 1899.
  799. 5499.  299.  599.  799. 2299. 2699. 3499.  349. 1499. 1999.  899.
 4999. 4999. 1899.  749.  499. 3299. 1599. 3299.  649. 3899. 4999. 2299.
 3299.  899. 2699. 3499. 1299. 5499. 3499. 3499.  899.  349.  999.  499.
 1499. 2199.  799.  299. 5499.  499. 1999. 2699.  999. 3499. 6999. 1499.
  999. 4999.  399. 4999.  499. 1499. 3899.  499. 4999. 1599.  899. 3499.
 5499. 5499. 1899. 5499. 2299. 1299.  899.  349. 1499. 3299. 5499. 1999.
  899.  299. 1499. 2499. 1899. 1599. 1999. 1999. 2699.  499. 3499.  649.
 1899. 3899. 1999. 6999.  499.  749. 4999. 1899.  899. 3899. 3499. 4999.
 1299.  349. 2699. 1599.  799. 2299.  399. 1599.  599. 2199.  999.  649.
 1299. 2499. 1299.  799. 1999. 6999. 1899. 3499. 2499.  499. 2699. 6999.
  499. 2199. 4999.  799.  499. 2699.  499. 5499. 4999.  399.  649. 2199.
  899. 1899.  999.  649. 1599.  399.  349. 2199. 2299. 3299.  399.  499.
  599.  499. 2199.  999. 3499. 1999. 3899.  599.  399.  499. 5499.  349.
 2199.  749. 4999.  299. 1999. 3499. 2499. 1599.  999. 1599. 4999.  499.
 2499.  799.  999. 2299. 1499. 1299. 1499.  749.  599. 1999. 2299. 3299.
  899.  299. 4999.  399. 5499.  349.  399.  499. 3299. 4999. 1499.  399.
  499.  799.  899. 3499.  749.  799. 2499.  999. 2199. 2499. 6999.  749.
  599. 2299. 2499. 2299.  349.  499. 1499. 3499.  899.  899. 1599. 4999.
  899.  499. 2499. 1499. 2699.  349. 6999.  899.  999.  599. 2499. 2699.
  799.  749. 1299.  899.  999.  649. 3899.  499.  799.  599. 3299. 1899.
 1899.  599. 3299.  349.  999. 6999. 1999.  749.  599. 1599. 3299.  299.
  799.  999. 6999.  799. 3299.  799. 1999.  749. 3299. 2499.  999. 4999.
 3899. 3899.  799. 2499. 5499. 6999.  749.  799.  649.  349.  999. 1899.
 2499. 6999. 1599.  899.  899.  599. 6999.  999. 1899. 2299.  999. 3299.
  299.  299.  499.  349. 5499.  649. 3499.  499.  499.  899. 2299.  899.
  649.  799.  499.  499.  799.  299.  299.  749. 1899. 4999.  349. 2299.
 2699.  799. 5499.  649. 3299. 5499.  749. 6999.  899. 2299. 1499.  399.
 1299.  399. 3499.  999. 2199.  599. 2299. 1599. 3499. 6999. 3899.  599.
  899. 4999. 3299.  899. 3499. 1599. 5499.  899. 2299.  799.  499.  799.
 1999. 2499. 1599. 3899.  499.  499.  799.  399. 3899. 2299.  649.  749.
 2499.  299.  899.  299.  499. 3899.  599.  749.  649.  649. 1499. 2299.]
```

In [14]:
```
print("Array type:", type(unit_price))
print("Array shape:", unit_price.shape)
print("Array dimensions:", unit_price.ndim)
```

```
Array type: <class 'numpy.ndarray'>
Array shape: (540,)
Array dimensions: 1
```

In [15]:
```
print("Total:", np.sum(unit_price))
print("Minimum:", np.min(unit_price))
print("Maximum:", np.max(unit_price))
print("Average:", np.mean(unit_price))
```

```
Total: 1125510.0
Minimum: 299.0
Maximum: 6999.0
Average: 2084.277777777778
```

In [16]:
```
print("By using array built in methods")
print("Total:", unit_price.sum())
print("Minimum:", unit_price.min())
print("Maximum:", unit_price.max())
print("Average:", unit_price.mean())
```

```
By using array built in methods
Total: 1125510.0
Minimum: 299.0
Maximum: 6999.0
Average: 2084.277777777778
```

In [17]:
```
column = df["UNIT_PRICE"]

print("Count:", column.count())
print("Total:", column.sum())
print("Minimum:", column.min())
print("Maximum:", column.max())
print("Average:", column.mean())
print("Median:", column.median())
print("Standard deviation:", column.std())
print("Variance:", column.var())
print("25th percentile:", column.quantile(0.25))
print("50th percentile:", column.quantile(0.50))
print("75th percentile:", column.quantile(0.75))
print("Unique values:", column.nunique())
print("Mode:", column.mode().tolist())
```

```
Count: 540
Total: 1125510.0
Minimum: 299.0
Maximum: 6999.0
Average: 2084.277777777778
Median: 1499.0
Standard deviation: 1777.271187130312
Variance: 3158692.8726035883
25th percentile: 749.0
50th percentile: 1499.0
75th percentile: 2849.0
Unique values: 25
Mode: [499.0]
```

In [18]:
```
duplicates = df[df.duplicated()]

print("Duplicate rows:")
print(duplicates)
```

```
Duplicate rows:
Empty DataFrame
Columns: [ORDER_ID, ORDER_DATE, CUSTOMER_ID, CUSTOMER_NAME, PRODUCT_ID, PRODUCT,
CATEGORY, QUANTITY, UNIT_PRICE, DISCOUNT_PCT, CITY, STATE, PAYMENT_METHOD, SALES_
CHANNEL, ORDER_STATUS, SHIPPING_FEE, CUSTOMER_SEGMENT]
Index: []
```

In [19]:
```
duplicates = df[df.duplicated(keep=False)]
```

```
print("All rows involved in duplication:")
print(duplicates)
```

```
All rows involved in duplication:
Empty DataFrame
Columns: [ORDER_ID, ORDER_DATE, CUSTOMER_ID, CUSTOMER_NAME, PRODUCT_ID, PRODUCT,
CATEGORY, QUANTITY, UNIT_PRICE, DISCOUNT_PCT, CITY, STATE, PAYMENT_METHOD, SALES_
CHANNEL, ORDER_STATUS, SHIPPING_FEE, CUSTOMER_SEGMENT]
Index: []
```

In [20]:
```
print("Duplicate row indexes:")
print(df.index[df.duplicated()].tolist())
```

```
Duplicate row indexes:
[]
```